

Einsendeaufgabe MET-E1: Metriken

1. Werkzeug: SonarQube – Statische Codeanalyse

Für diese Aufgabe habe ich mein Projekt ReconForge mit SonarQube analysiert. ReconForge ist ein C#/.NET-Projekt aus meinem Softwaretechnik-Projekt.

Ich habe SonarQube lokal mit Docker gestartet, weil die manuelle Installation aufwendiger sein kann. Danach habe ich ReconForge mit dem SonarScanner für .NET untersucht.

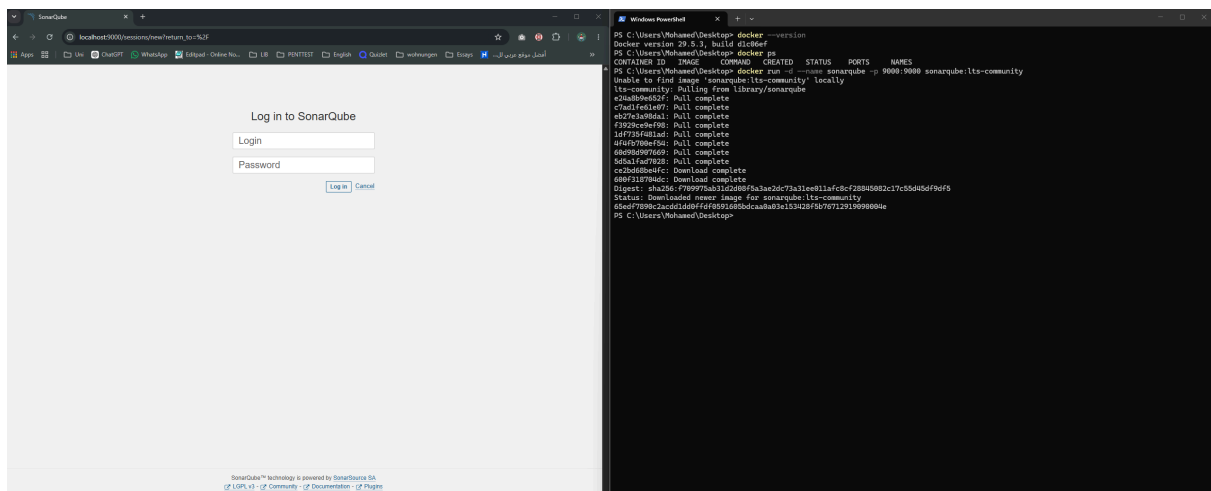


Abbildung 1: Start von SonarQube mit Docker

Hier habe ich SonarQube mit Docker gestartet. Danach war die lokale SonarQube-Oberfläche über localhost:9000 erreichbar.

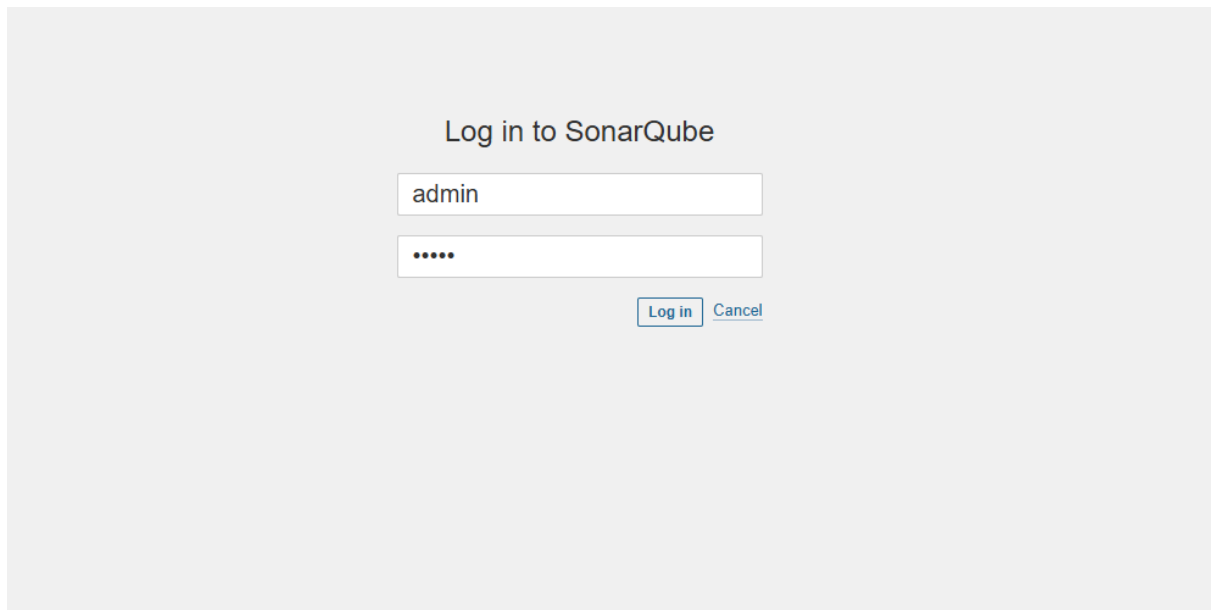


Abbildung 2: Anmeldung an SonarQube

Nach dem Start habe ich mich an der lokalen SonarQube-Oberfläche angemeldet.

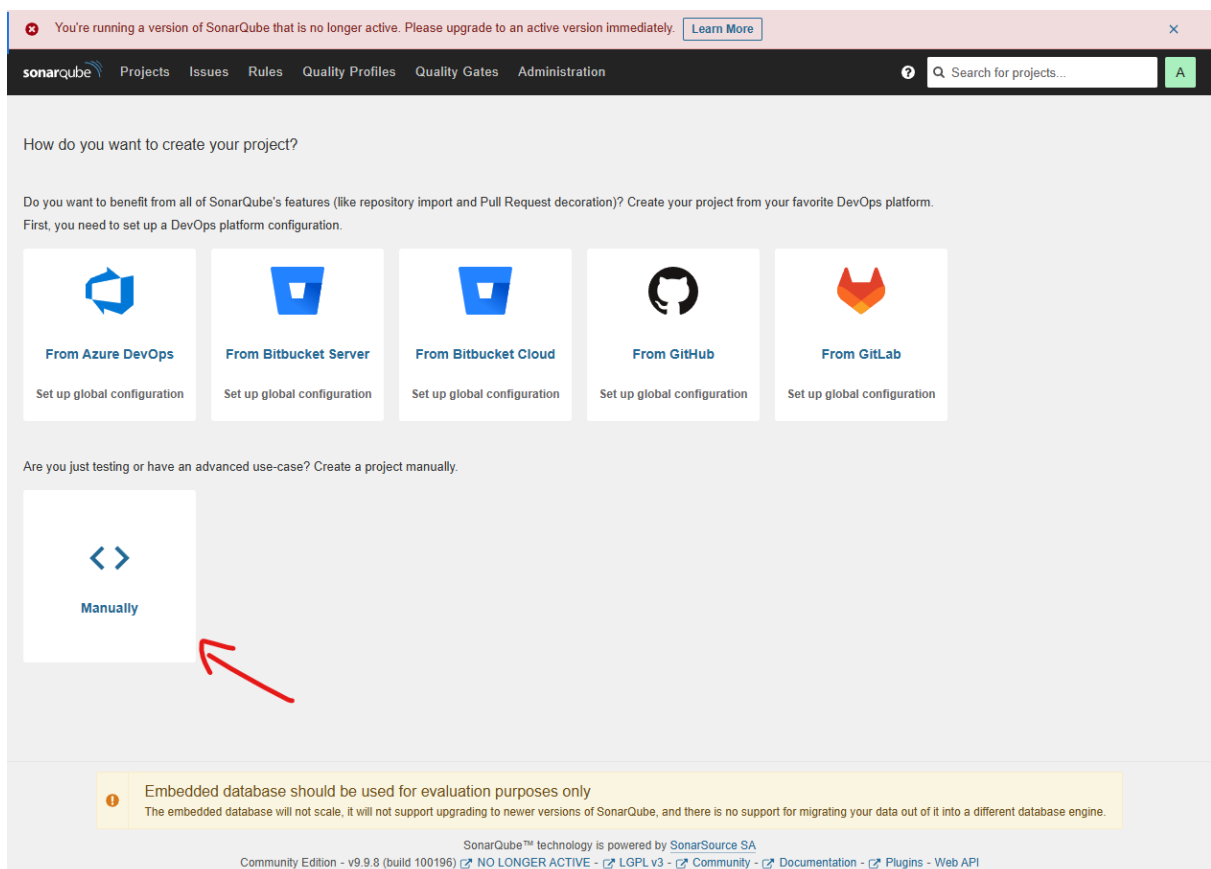


Abbildung 3: Projekt manuell anlegen

Ich habe das Projekt manuell angelegt, weil ich ReconForge lokal analysieren wollte.

sonarcube Projects Issues Rules Quality Profiles Quality Gates Administration

Search for projects... A

Create a project

All fields marked with * are required

Project display name *

ReconForge ✓

Up to 255 characters. Some scanners might override the value you provide.

Project key *

ReconForge ✓

The project key is a unique identifier for your project. It may contain up to 400 characters. Allowed characters are alphanumeric, '-' (dash), '_' (underscore), '.' (period) and ':' (colon), with at least one non-digit.

Main branch name *

main

The name of your project's default branch [Learn More](#)

Set Up

Abbildung 4: Projekt ReconForge erstellen

Hier habe ich den Projektnamen und den Project Key „ReconForge“ eingetragen. Als Hauptbranch wurde „main“ verwendet.

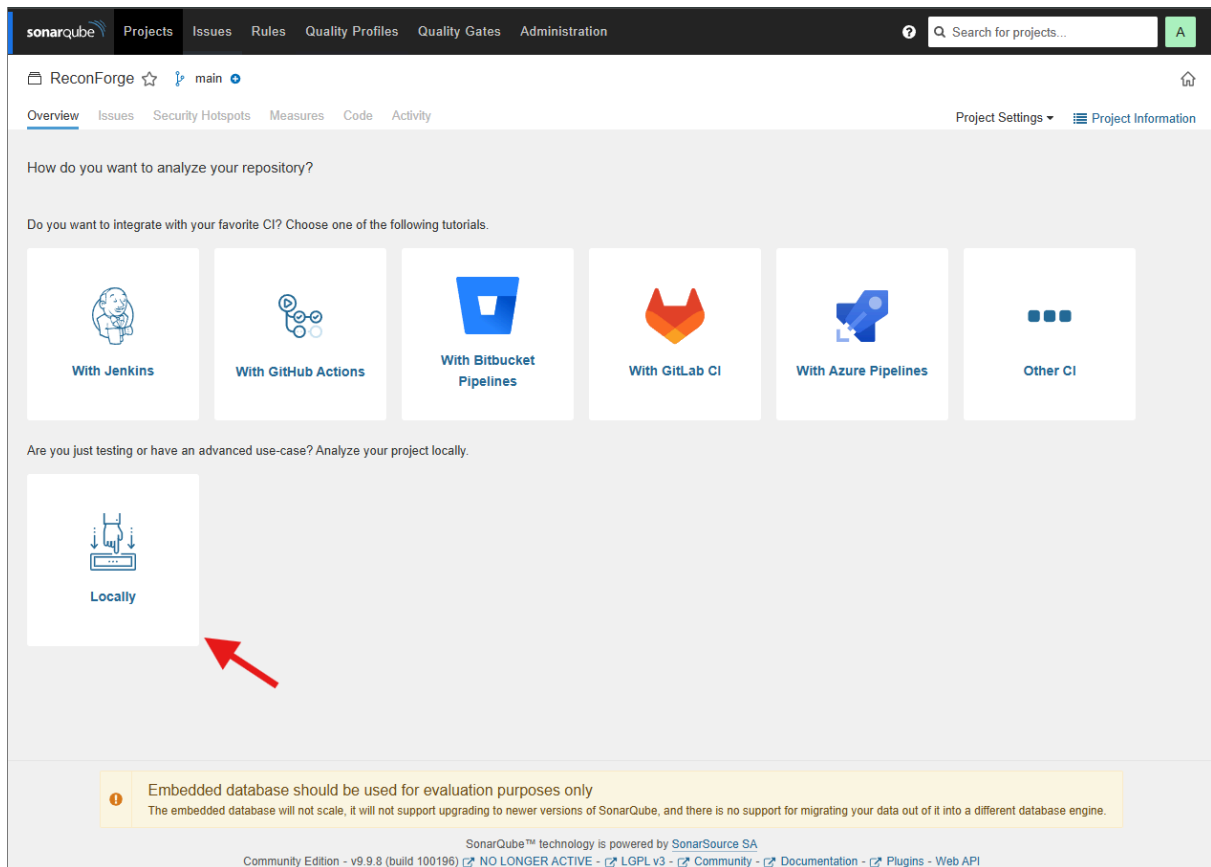


Abbildung 5: Lokale Analyse auswählen

Für die Analyse habe ich „Locally“ ausgewählt, da SonarQube und ReconForge lokal auf meinem Rechner laufen.

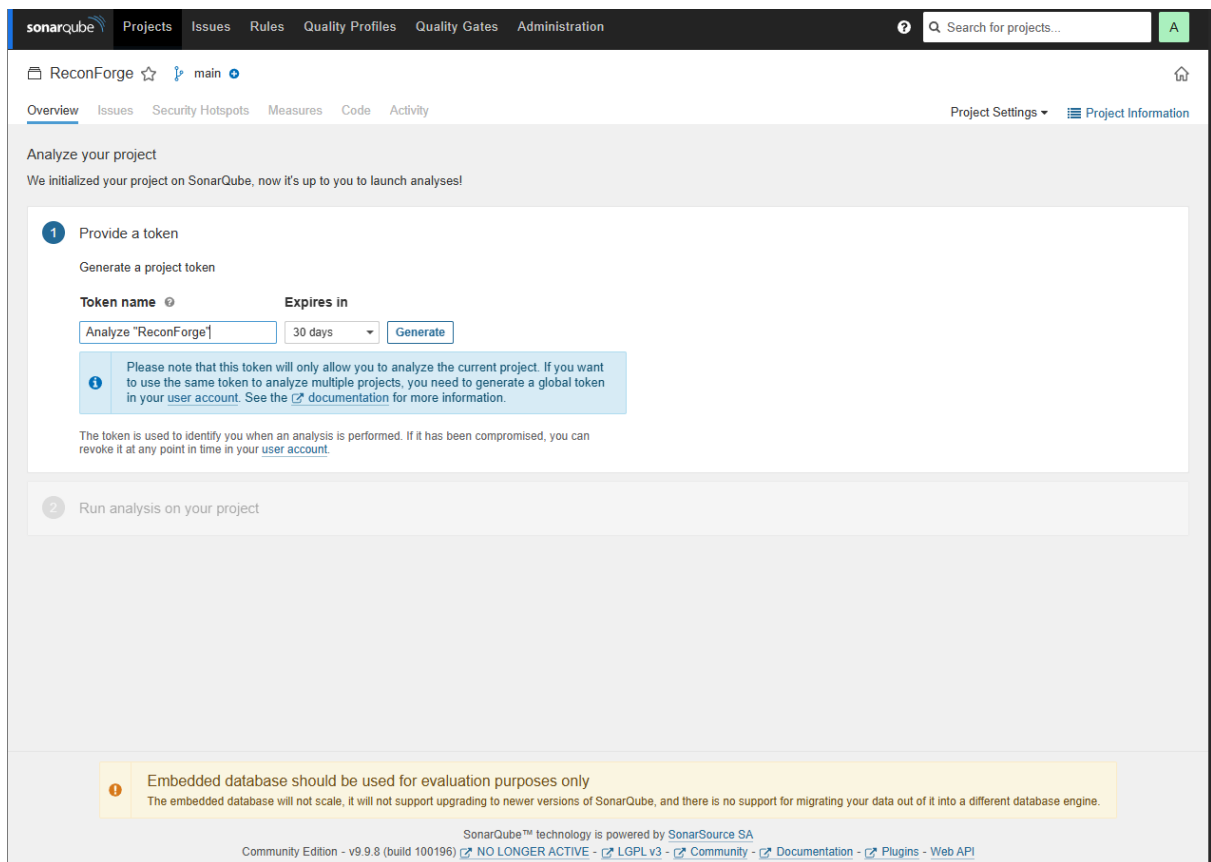


Abbildung 6: Analyse-Token erstellen

Für die lokale Analyse wurde ein Projekt-Token erstellt. Dieses Token wurde später vom SonarScanner verwendet.

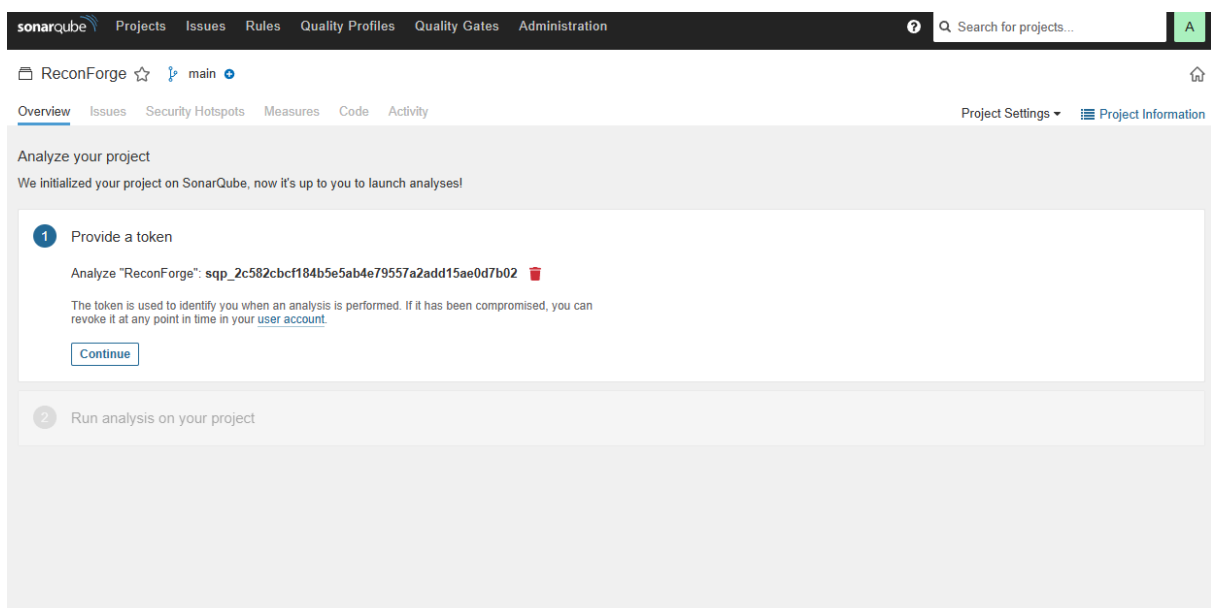


Abbildung 7: Erzeugtes Analyse-Token

Nach dem Klick auf „Generate“ wurde das Token erstellt. Es ist sichtbar, weil es nur für eine lokale Testinstallation verwendet wurde.

The screenshot displays the SonarQube web interface for a project named 'ReconForge'. The top navigation bar includes links for Projects, Issues, Rules, Quality Profiles, Quality Gates, and Administration. The main content area shows a progress bar with two steps: 'Provide a token' (completed) and 'Run analysis on your project' (current step). Under 'Run analysis on your project', there are options for build tools (Maven, Gradle, .NET, Other) and scanner types (.NET Core, .NET Framework). It provides a command to install the sonarscanner globally and instructions on how to execute the scanner. A footer note mentions that the embedded database is for evaluation purposes only.

Abbildung 8: Analyse für .NET vorbereiten

Da ReconForge ein .NET-Projekt ist, habe ich „.NET“ und „.NET Core“ ausgewählt. SonarQube zeigte danach die passenden Befehle für die Analyse.

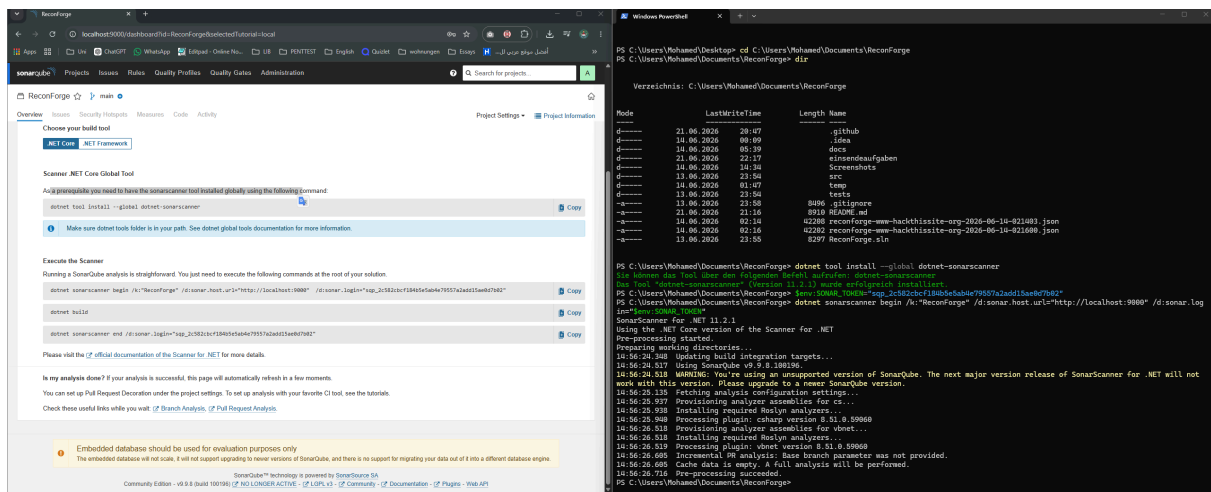


Abbildung 9: SonarScanner installieren und Analyse starten

Im Projektordner habe ich den SonarScanner for .NET installiert und die Analyse mit `dotnet sonarscanner begin` gestartet.

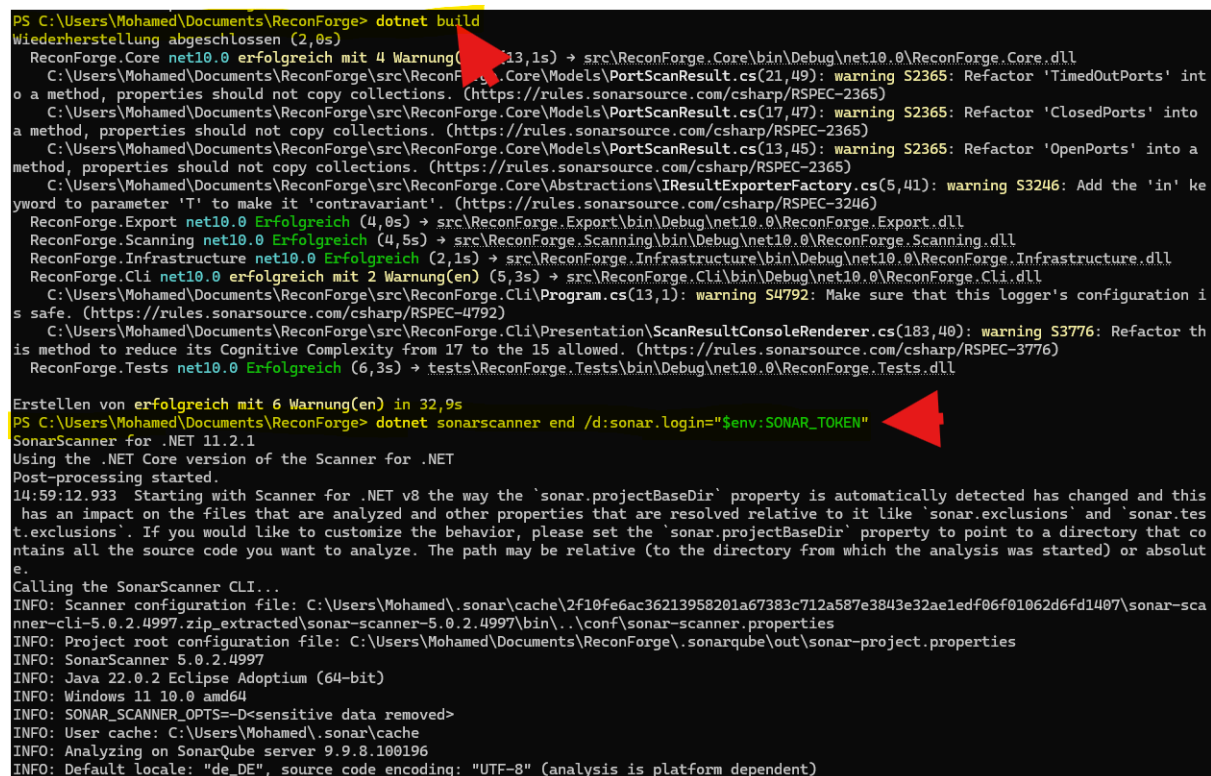


Abbildung 10: Build und Abschluss der Analyse

Danach habe ich das Projekt mit `dotnet build` gebaut und die Analyse mit `dotnet sonarscanner end` abgeschlossen.

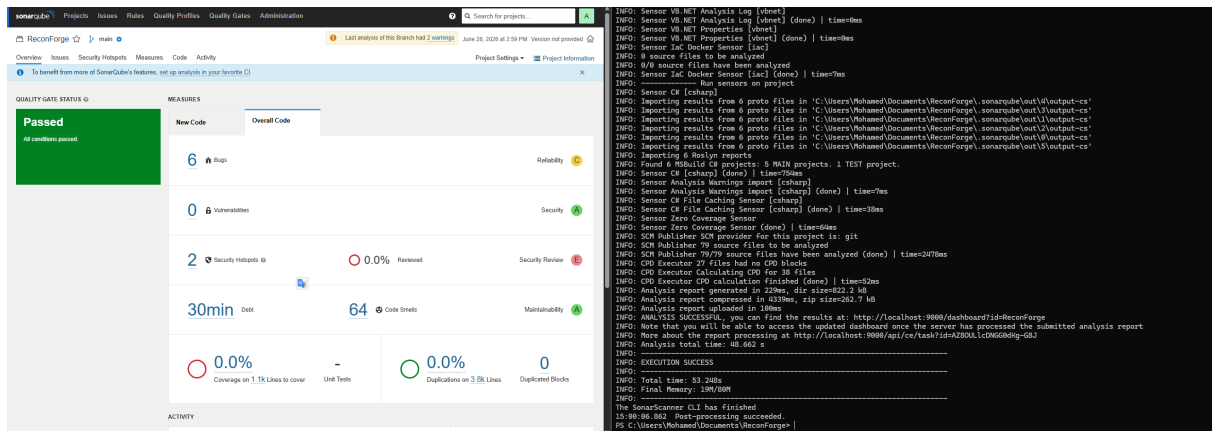


Abbildung 11: Erfolgreiche SonarQube-Analyse

Die Analyse wurde erfolgreich abgeschlossen. Rechts sieht man die Meldung „EXECUTION SUCCESS“.

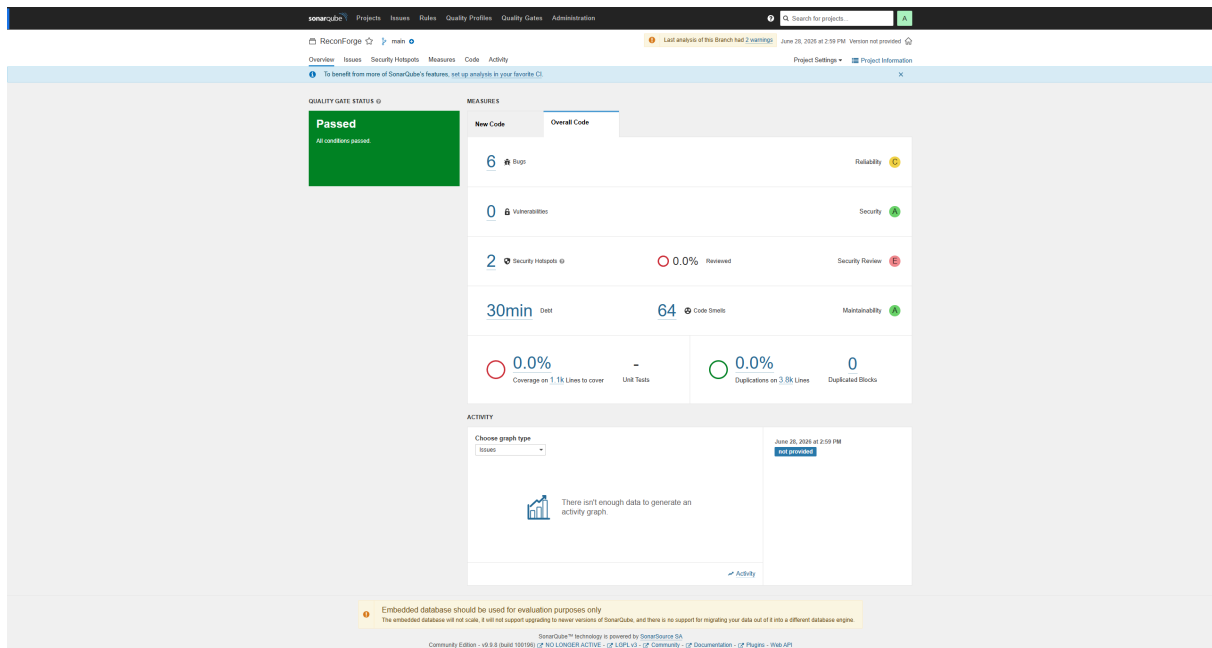


Abbildung 12: SonarQube-Dashboard nach der Analyse

Das Quality Gate wurde bestanden. SonarQube zeigte 6 Bugs, 0 Vulnerabilities, 2 Security Hotspots, 64 Code Smells, 30 Minuten technische Schuld, 0,0 % Coverage und 0,0 % Duplikationen.

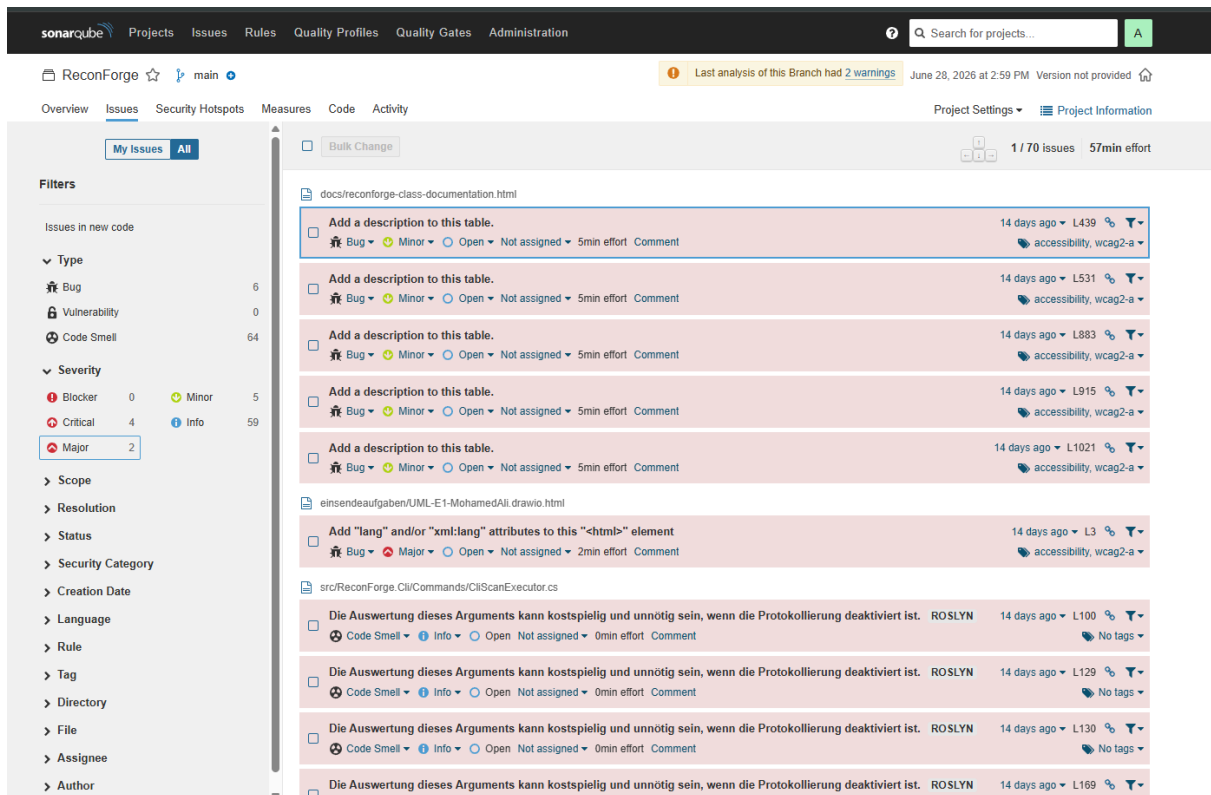


Abbildung 13: Gefundene Issues in SonarQube

In der Issues-Ansicht wurden die gefundenen Probleme angezeigt. Insgesamt wurden 70 Issues gefunden.

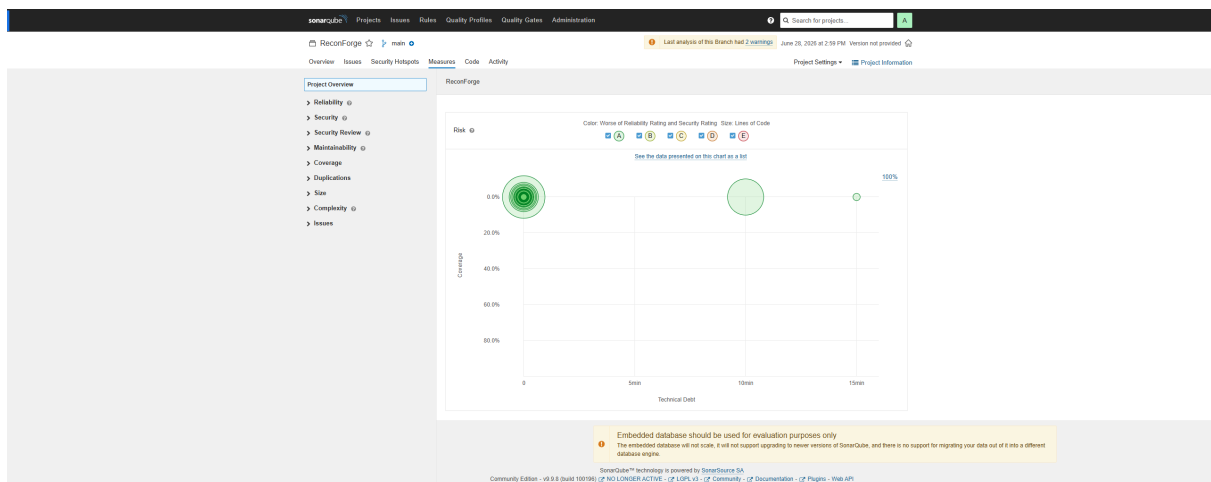


Abbildung 14: Measures-Ansicht in SonarQube

In der Measures-Ansicht werden die Metriken grafisch dargestellt. Die Coverage liegt bei 0,0 %, weil keine Test-Coverage-Daten übergeben wurden.

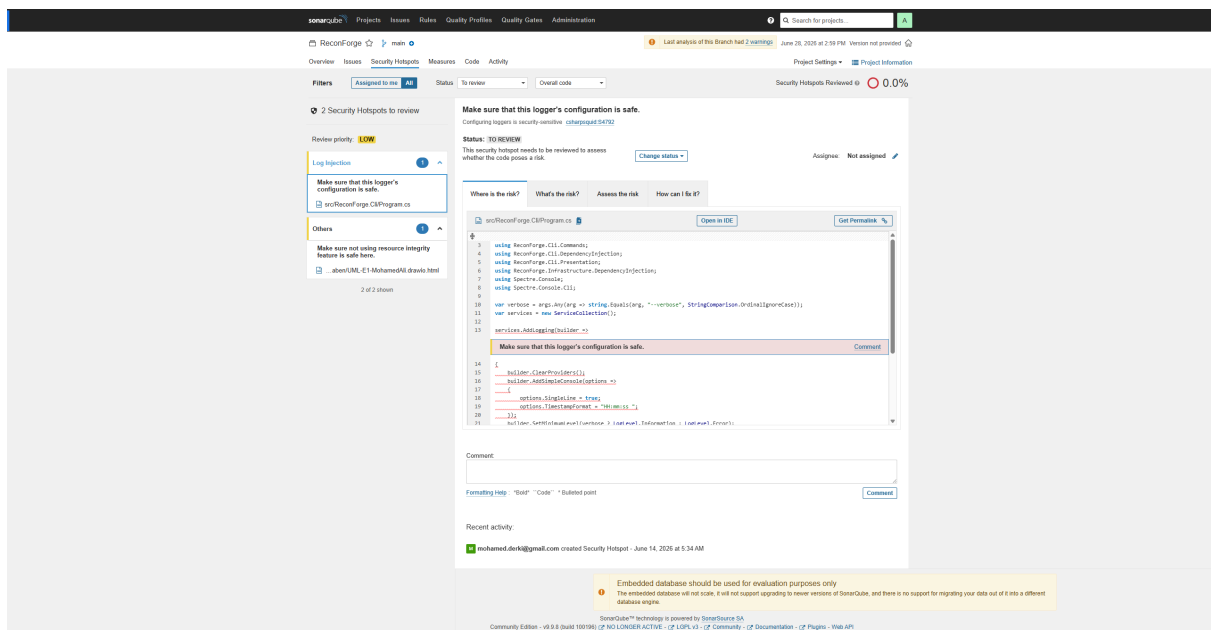


Abbildung 15: Security Hotspots in SonarQube

SonarQube zeigte zwei Security Hotspots. Diese Stellen sollten manuell geprüft werden.

2. Werkzeug: Visual Studio – Code Metrics

Als zweites Werkzeug habe ich die Code-Metrics-Funktion von Visual Studio verwendet. Damit können Metriken wie Wartbarkeit, zyklomatische Komplexität, Kopplung und Codezeilen für ein .NET-Projekt berechnet werden.

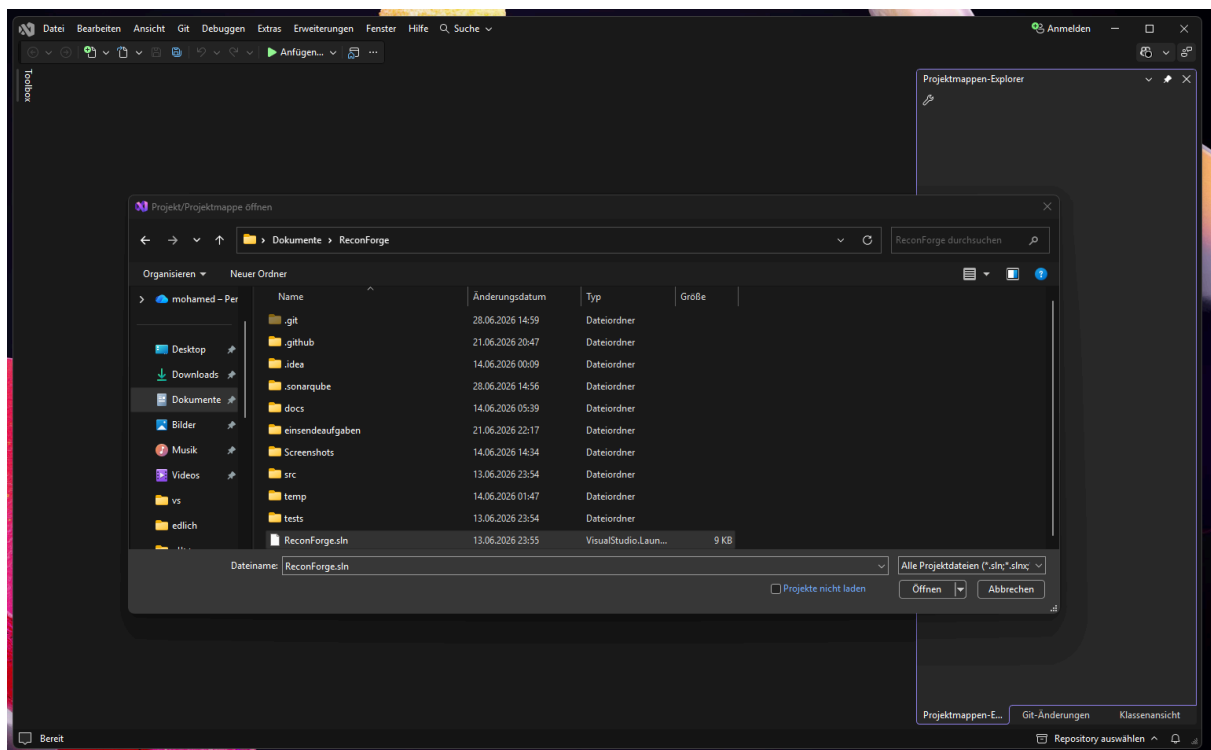


Abbildung 16: ReconForge in Visual Studio öffnen

Hier habe ich die Projektmappe ReconForge.sln in Visual Studio geöffnet, damit das Projekt analysiert werden kann.

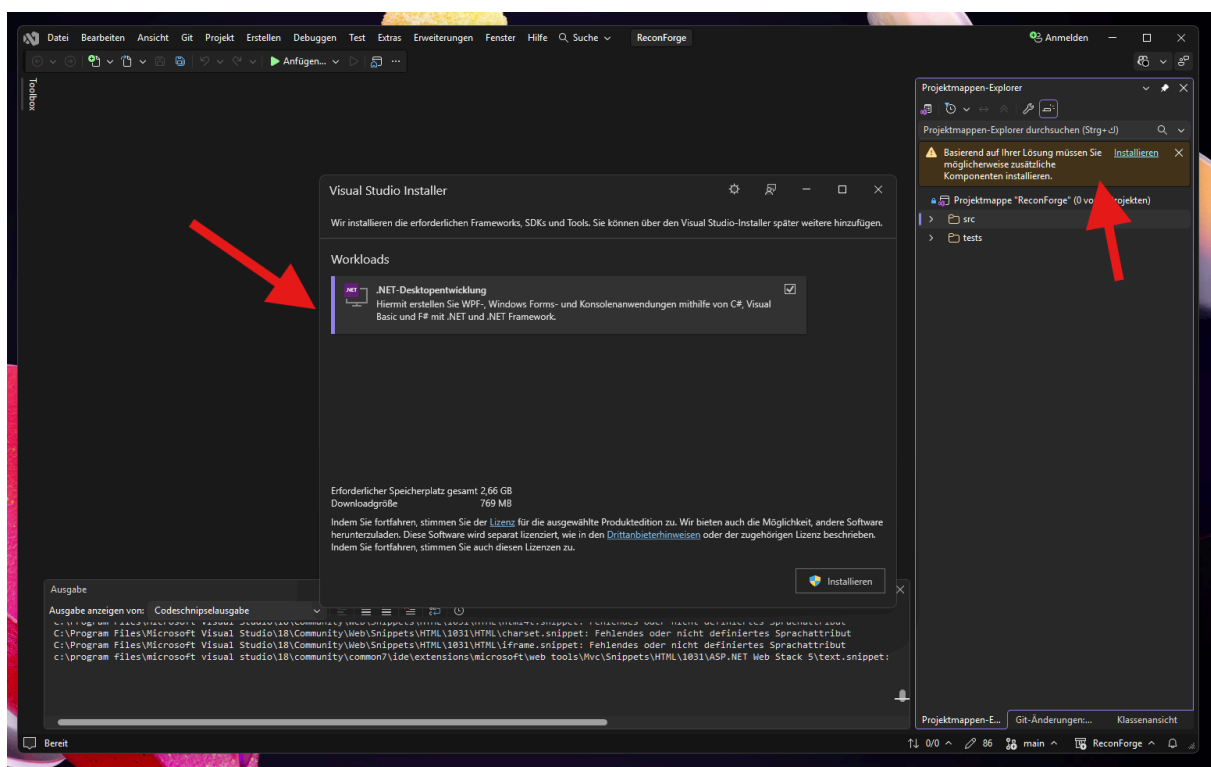


Abbildung 17: Fehlende Visual-Studio-Komponenten installieren

Visual Studio hat erkannt, dass noch zusätzliche Komponenten benötigt werden. Deshalb wurde die .NET-Desktopentwicklung installiert.

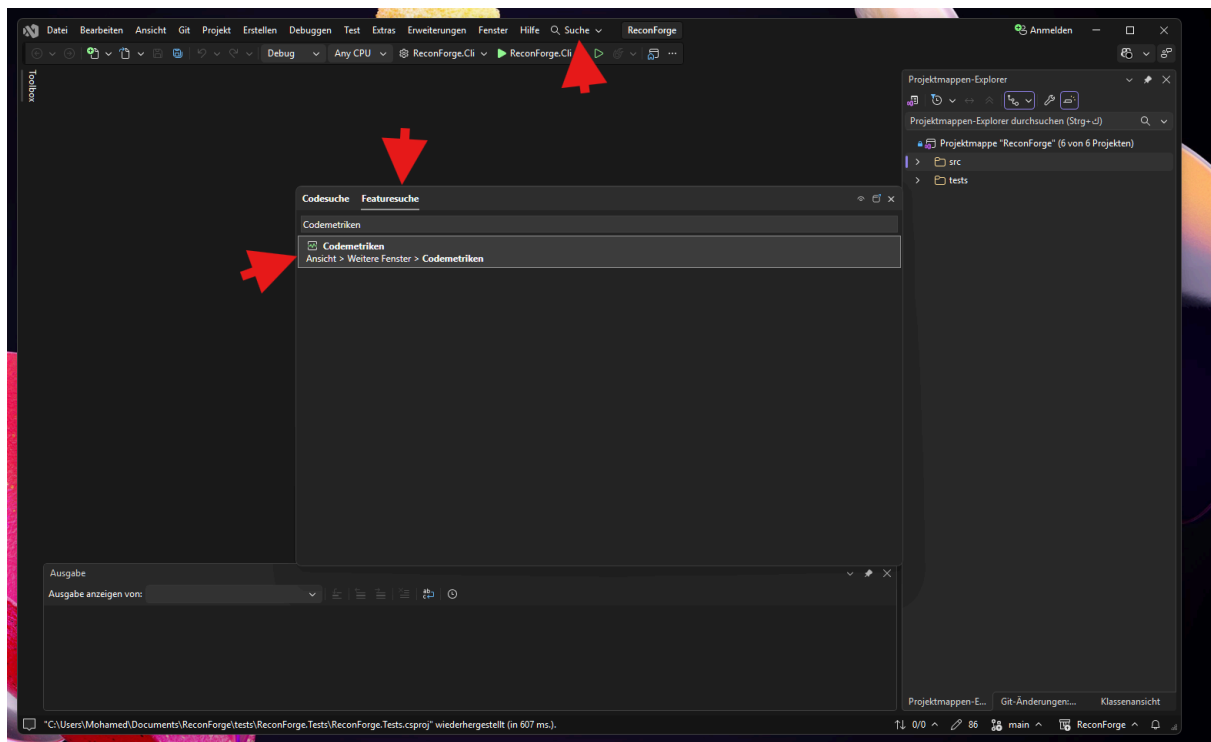


Abbildung 18: Codemetriken in Visual Studio suchen

Über die Featuresuche habe ich nach „Codemetriken“ gesucht, um die Code-Metrics-Funktion zu öffnen.

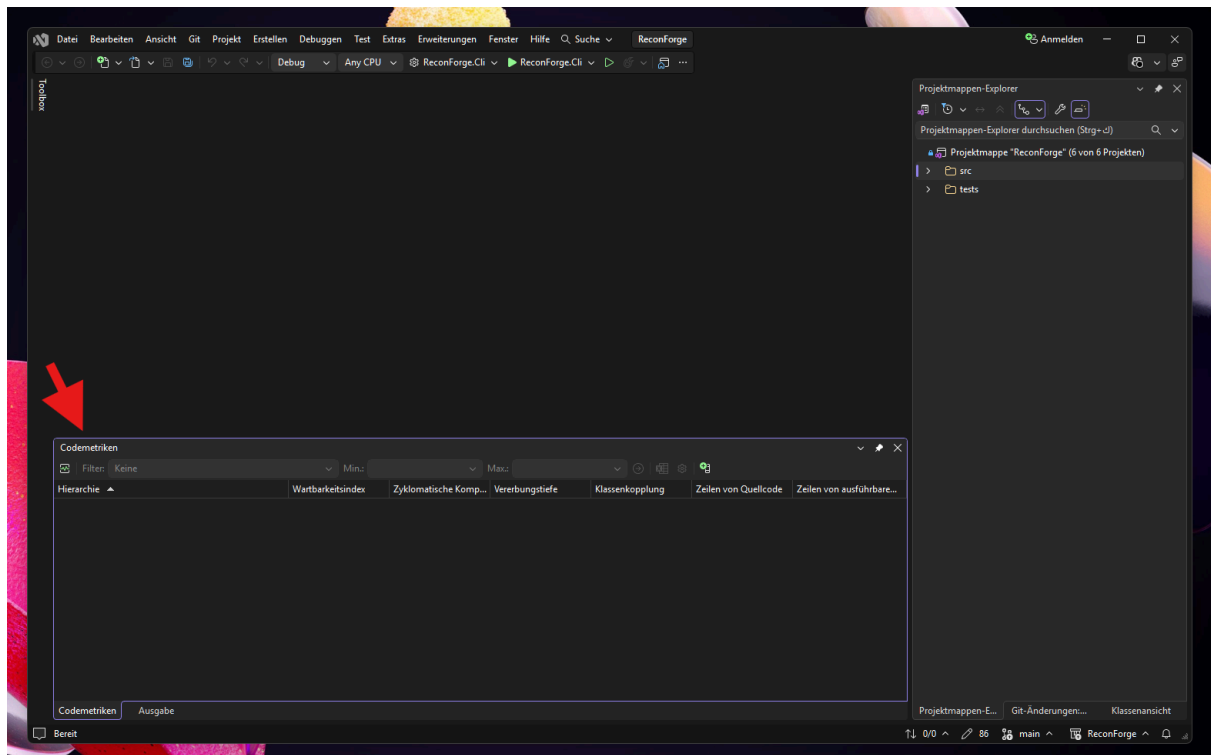


Abbildung 19: Codemetriken-Fenster in Visual Studio

Hier wurde das Codemetriken-Fenster geöffnet. Die Tabelle war zuerst leer, weil die Metriken noch berechnet werden mussten.

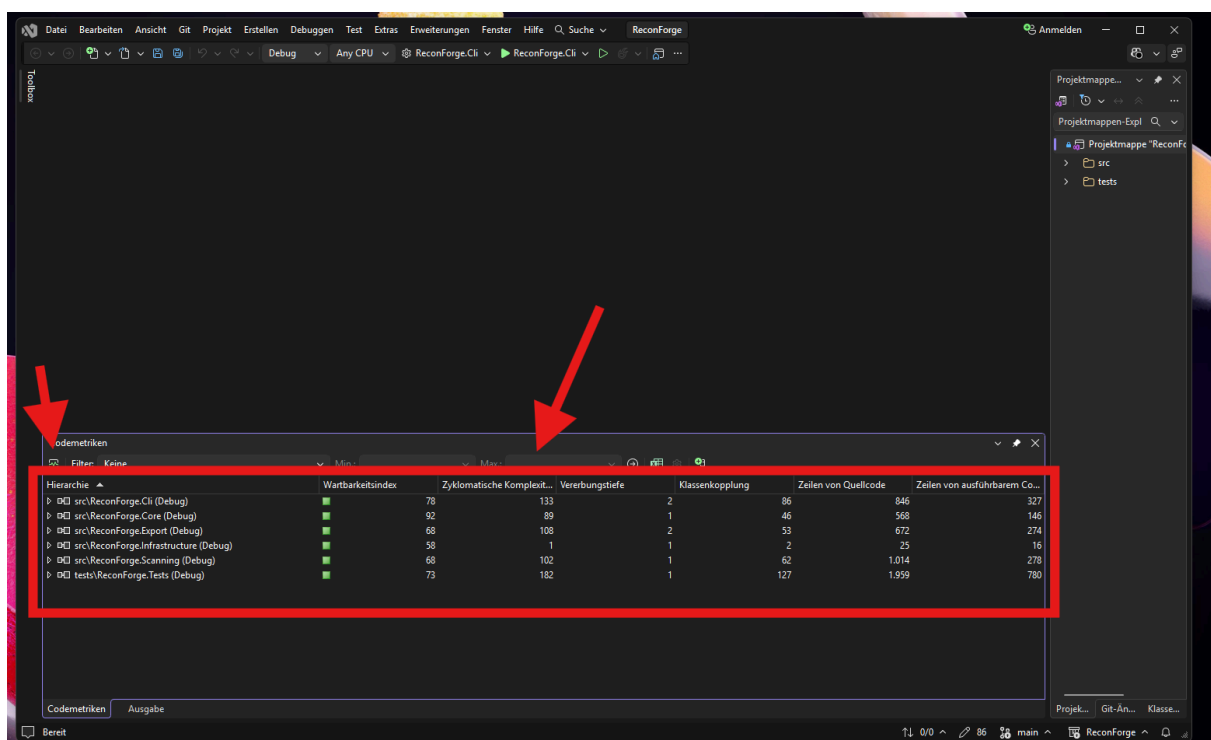


Abbildung 20: Ergebnis der Codemetriken in Visual Studio

Hier sieht man die berechneten Codemetriken für ReconForge. Visual Studio zeigt Wartbarkeitsindex, Komplexität, Kopplung und Codezeilen für die einzelnen Projekte an.

Die Ergebnisse zeigen, dass Visual Studio eher klassische Code-Metriken berechnet. Für mich war dieses Werkzeug einfacher zu verwenden als SonarQube, weil es direkt in Visual Studio integriert ist. Dafür zeigt es weniger Hinweise zu Bugs, Security Hotspots oder technischer Schuld.

3. Kleine manuelle Metrik-Berechnung

Zusätzlich zu den beiden Werkzeugen habe ich eine kleine Metrik beispielhaft von Hand berechnet. Dafür habe ich die Methode `TryExtractDomain` aus der Datei `src/ReconForge.Core/Validation/DomainNameValidator.cs` gewählt.

```
private static bool TryExtractDomain(
    string input,
    out string domain,
    out string errorMessage)
{
    domain = string.Empty;
    errorMessage = string.Empty;

    if (input.Contains("://", StringComparison.Ordinal))
    {
        if (!Uri.TryCreate(input, UriKind.Absolute, out var uri))
        {
            errorMessage = "The domain input is not a valid URL.";
            return false;
        }

        if (uri.Scheme is not ("http" or "https"))
        {
            errorMessage = "Only plain domains or http/https URLs are supported.";
            return false;
        }
    }
}
```

```

    if (string.IsNullOrEmpty(uri.Host))
    {
        errorMessage = "The URL does not contain a valid host.";
        return false;
    }

    domain = uri.Host;
    return true;
}

if (input.Contains('/'))
{
    var withoutTrailingSlash = input.TrimEnd('/');
    if (withoutTrailingSlash.Contains('/'))
    {
        errorMessage = "Use a domain name without a path, or provide a full http/https URL.";
        return false;
    }

    domain = withoutTrailingSlash;
    return true;
}

domain = input;
return true;
}

```

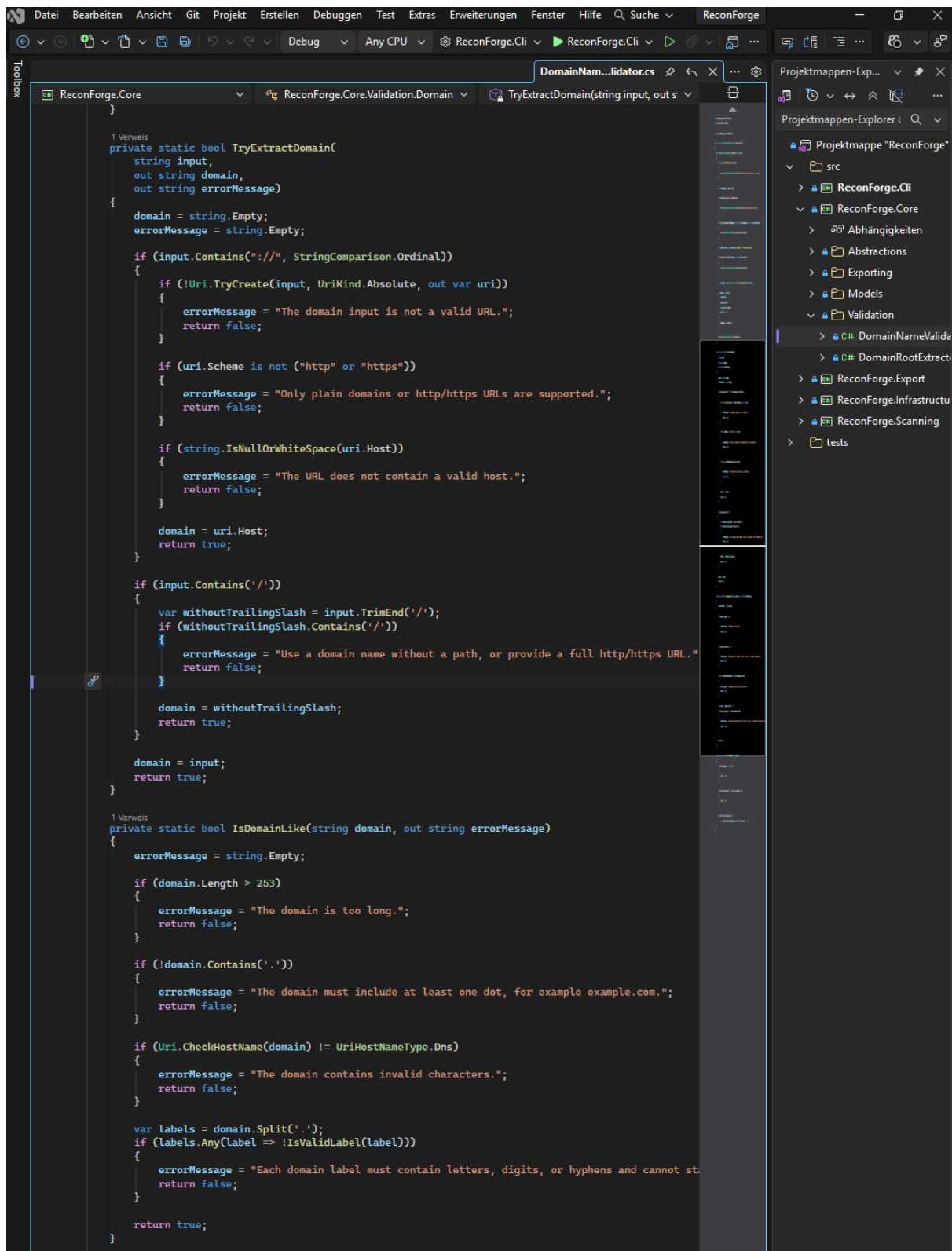


Abbildung 21: Methode TryExtractDomain für die manuelle McCabe-Berechnung

Diese Methode eignet sich für die Berechnung, weil sie mehrere if-Abfragen enthält.

Für die vereinfachte McCabe-Berechnung verwende ich die Regel:

M McCabe-Komplexität = 1 + Anzahl der Entscheidungspunkte

Entscheidungspunkte in der Methode:

1. `if (input.Contains("://", StringComparison.Ordinal))`
2. `if (!Uri.TryCreate(...))`
3. `if (uri.Scheme is not ("http" or "https"))`
4. `if (string.IsNullOrEmpty(uri.Host))`
5. `if (input.Contains('/'))`
6. `if (withoutTrailingSlash.Contains('/'))`

Damit ergibt sich:

$$1 + 6 = 7$$

Die vereinfachte McCabe-Komplexität der Methode beträgt also 7. Die Methode hat mehrere mögliche Ausführungspfade, bleibt aber noch gut nachvollziehbar.

Hinweis: Ich habe hier bewusst eine vereinfachte Berechnung verwendet und die if-Abfragen gezählt. Je nach genauer Zählweise könnten auch einzelne logische Ausdrücke zusätzlich berücksichtigt werden.

Vergleich der Werkzeuge

SonarQube liefert viele Hinweise zur Codequalität, zum Beispiel Bugs, Code Smells, Security Hotspots, Duplikationen und technische Schuld. Die Einrichtung war aber etwas aufwendiger, weil Docker und der SonarScanner konfiguriert werden mussten.

Visual Studio Metrics war einfacher zu starten und zeigt vor allem technische Metriken wie Wartbarkeit, Komplexität, Kopplung und Codezeilen. Beide Werkzeuge ergänzen sich gut, weil sie unterschiedliche Schwerpunkte haben.

Fazit

Durch beide Werkzeuge konnte ich den Zustand meines Projekts besser einschätzen. SonarQube war hilfreich, um konkrete Probleme im Code zu finden. Visual Studio Code Metrics war nützlich, um die Struktur und Komplexität der einzelnen Projekte zu vergleichen.

Ich habe gelernt, dass Metriken den Code nicht automatisch verbessern, aber gute Hinweise geben, welche Stellen man genauer anschauen sollte.